

WEEK – 5(Skill)

Name: Anthony Nissy

Rollno: 2520030064

Section: 8

Experiment No. 9 (Q9)

Title: Directory Navigation (cd) and Built-in Command Dispatch in a Custom Shell

Aim:

To implement directory navigation functionality (similar to the cd command) that changes directories, validates paths, updates the working directory, handles errors, and maintains the previous directory. To identify shell built-in commands, create a dispatch table, execute built-in commands in-process, handle invalid commands, maintain shell state, and test the dispatch logic.

Objective:

Part 1 – Directory Navigation (cd)

- To change the current working directory using chdir().
- To validate the target path before changing directories.
- To update the working directory after a successful change.
- To handle errors such as invalid or inaccessible paths.
- To maintain the previous directory for use with "cd -".
- To test directory navigation with valid and invalid paths.

Part 2 – Built-in Command Dispatcher

- To identify commands that must be treated as shell built-ins.
- To create a dispatch table mapping command names to handler functions.
- To execute built-in commands in-process (without fork()).
- To handle invalid or unrecognized commands gracefully.
- To maintain shell state such as the command count and current directory.
- To test the dispatch logic with valid and invalid commands.

Theory:

1. Directory Navigation and the cd Command

Every process has a current working directory associated with it. The cd command changes this directory for the shell process using the chdir() system call. Because cd modifies the shell's own state, it must be implemented as a built-in command rather than an external program — an external program running in a child process cannot change the parent shell's working directory.

2. chdir()

The chdir() system call changes the calling process's current working directory to the specified path.

```
int chdir(const char *path);
```

It returns 0 on success and -1 on failure, setting errno to indicate the cause (e.g. ENOENT for a missing path, ENOTDIR if the path is not a directory).

3. Path Validation

Before changing directories, the target path is validated using stat(). This confirms that the path exists and that it refers to a directory, preventing chdir() from being called on an invalid or non-directory path.

```
struct stat st;  
stat(path, &st);  
S_ISDIR(st.st_mode) /* true if path is a directory */
```

4. Maintaining the Previous Directory

A well-behaved cd implementation stores the directory the shell was in before the change, similar to the OLDPWD variable in real shells. This allows the special argument "cd -" to return to the previous directory.

5. Error Handling

If the target directory does not exist, is not a directory, or chdir() fails for any other reason, an appropriate error message is displayed and the current directory remains unchanged.

6. Shell Built-in Commands

Built-in commands are commands executed directly by the shell process instead of being launched as external programs via `fork()` and `exec()`. Examples include `cd`, `pwd`, `exit` and `help`. They are used for operations that must affect the shell's own state (such as the working directory) or that need to run without the overhead of creating a new process.

7. Dispatch Table

A dispatch table is an array that maps each built-in command name to a corresponding handler function (a function pointer). When a command is entered, the shell searches the table for a matching name and calls the associated function, avoiding a long chain of if-else statements.

```
typedef struct {
    const char *name;
    void (*func)(char *);
} DispatchEntry;
```

8. In-Process Execution

Because built-in commands run in-process, they execute immediately within the shell and can directly modify shell state (such as `currDirectory` or `prevDirectory`) without any inter-process communication.

9. Handling Invalid Commands

If the entered command name does not match any entry in the dispatch table, the shell reports "command not found" instead of crashing or silently ignoring the input.

Algorithm:

Part 1 – Directory Navigation

- Start the program.
- Record the starting working directory using `getcwd()`.
- Read the target directory argument (or `"-"` for the previous directory).
- If no argument is given, use the `HOME` directory.
- Validate the target path using `stat()` and `S_ISDIR()`.
- If validation fails, display an error and stop.
- Save the current directory into `prevDirectory`.
- Call `chdir()` to change to the target directory.
- Update `currDirectory` using `getcwd()`.
- Display the new working directory.
- Test navigation with a valid path, `"cd -"`, and an invalid path.

Part 2 – Built-in Command Dispatcher

- Start the program.
- Build a dispatch table of {command name, handler function} pairs.
- Read a command and its argument from the user.
- Search the dispatch table for a matching command name.
- If found, increment the command counter and call the handler function.
- If not found, display "command not found".
- Repeat until the exit command is entered.
- Stop the program.

Functions Used:

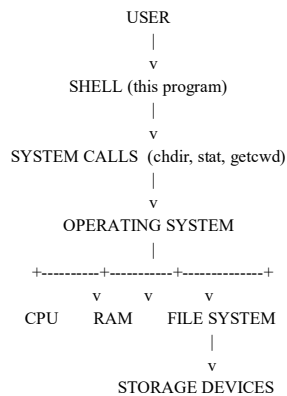
Function	Purpose
<code>main()</code>	Starting point of the program
<code>chdir()</code>	Changes the current working directory
<code>getcwd()</code>	Retrieves the current working directory
<code>stat()</code>	Retrieves file/directory information for path validation
<code>S_ISDIR()</code>	Checks whether a <code>stat()</code> result refers to a directory
<code>getenv()</code>	Retrieves the <code>HOME</code> environment variable
<code>changeDirectory()</code>	Implements <code>cd</code> , including <code>"cd -"</code> and error handling
<code>updateWorkingDirectory()</code>	Refreshes the stored current directory
<code>isBuiltin()</code>	Checks whether a command exists in the dispatch table

executeBuiltin()	Looks up and calls the handler for a built-in command
builtin_cd() / builtin_pwd() / builtin_exit() / builtin_help()	Handler functions for built-in commands
fgets()	Reads command input
sscanf()	Splits input into command and argument
printf()	Displays output and messages

Hardware and Operating System Relationship

Hardware consists of physical components such as the CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship



How OS Interacts with Hardware

- **Process Management** – The OS manages the shell process and any commands it runs.
- **File Management** – The OS resolves paths and verifies directories via the file system.
- **Memory Management** – The OS manages memory used to store shell state.
- **Device Management** – The OS controls the terminal used for input and output.
- **Security** – The OS enforces directory and file access permissions.

Observation

The program successfully changes the current working directory using `chdir()`, validates paths with `stat()` before switching, updates and displays the new working directory, and reports clear errors for invalid paths. It also stores the previous directory so that "cd -" correctly returns to it.

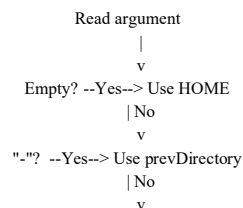
The built-in dispatcher correctly identifies recognized commands (`cd`, `pwd`, `help`, `exit`) using the dispatch table, executes their handler functions in-process, keeps count of the commands dispatched, and reports "command not found" for unrecognized input.

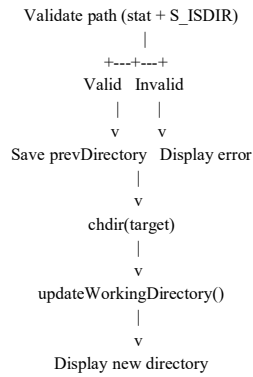
Result

Directory navigation functionality similar to the `cd` command was successfully implemented, including path validation, working-directory updates, error handling, and maintenance of the previous directory. A built-in command dispatcher was also successfully implemented to identify built-ins, execute them in-process using a dispatch table, handle invalid commands, and maintain shell state.

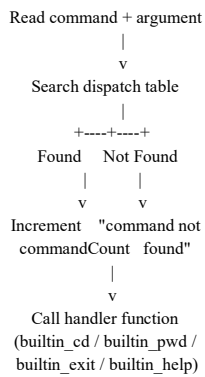
Design / Flow

Directory Navigation Flow





Dispatch Logic Flow



CODE-

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

```

```

#define MAX 200

```

```

char prevDirectory[MAX] = "";
char currDirectory[MAX] = "";

```

```

/* ----- Part 1: Directory Navigation ----- */

```

```

int validatePath(const char *path)
{
    struct stat st;
    if (stat(path, &st) != 0)
    {
        printf("cd: %s: No such file or directory\n", path);
        return 0;
    }
    if (!S_ISDIR(st.st_mode))
    {
        printf("cd: %s: Not a directory\n", path);
        return 0;
    }
    return 1;
}

```

```

void updateWorkingDirectory()
{
    if (getcwd(currDirectory, sizeof(currDirectory)) == NULL)
        perror("getcwd");
}

```

```

void changeDirectory(const char *arg)
{
    char target[MAX];

```

```

    if (arg == NULL || strlen(arg) == 0)
    {
        char *home = getenv("HOME");
        if (home == NULL)
        {
            printf("cd: HOME not set\n");
            return;
        }
        strncpy(target, home, MAX);
    }
    else if (strcmp(arg, ".") == 0)
    {
        if (strlen(prevDirectory) == 0)
        {
            printf("cd: OLDPWD not set\n");
            return;
        }
        strncpy(target, prevDirectory, MAX);
        printf("%s\n", target);
    }
    else
    {
        strncpy(target, arg, MAX);
    }

```

```

    if (!validatePath(target))
        return;

```

```

    strncpy(prevDirectory, currDirectory, MAX);

```

```

    if (chdir(target) != 0)
    {
        perror("cd");
        return;
    }

```

```

    updateWorkingDirectory();
    printf("Directory changed to: %s\n", currDirectory);
}

```

```

void testNavigation()
{
    printf("\n--- Testing Directory Navigation ---\n");
    updateWorkingDirectory();
    printf("Starting directory: %s\n", currDirectory);

    changeDirectory("/tmp");
    changeDirectory(".");
    changeDirectory("/nonexistent_dir_xyz");
}

```

```

/* ----- Part 2: Built-in Command Dispatcher ----- */

```

```

int commandCount = 0;

```

```

void builtin_cd(char *arg) { changeDirectory(arg); }

```

```

void builtin_pwd(char *arg)
{
    (void)arg;
    updateWorkingDirectory();
    printf("%s\n", currDirectory);
}

void builtin_exit(char *arg)
{
    (void)arg;
    printf("Exiting shell...\n");
    exit(0);
}

void builtin_help(char *arg)
{
    (void)arg;
    printf("Built-in commands: cd, pwd, exit, help\n");
}

typedef struct {
    const char *name;
    void (*func)(char *);
} DispatchEntry;

DispatchEntry dispatchTable[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"exit", builtin_exit},
    {"help", builtin_help}
};

int dispatchTableSize = sizeof(dispatchTable) / sizeof(DispatchEntry);

int isBuiltin(const char *cmd)
{
    for (int i = 0; i < dispatchTableSize; i++)
        if (strcmp(cmd, dispatchTable[i].name) == 0)
            return 1;
    return 0;
}

void executeBuiltin(char *cmd, char *arg)
{
    commandCount++;
    for (int i = 0; i < dispatchTableSize; i++)
    {
        if (strcmp(cmd, dispatchTable[i].name) == 0)
        {
            dispatchTable[i].func(arg);
            return;
        }
    }
    printf("%s: command not found\n", cmd);
}

void testDispatchLogic()
{
    printf("\n--- Testing Dispatch Logic ---\n");
    executeBuiltin("pwd", NULL);
    executeBuiltin("cd", "/tmp");
    executeBuiltin("help", NULL);
    executeBuiltin("foo", NULL);
    printf("Total commands dispatched: %d\n", commandCount);
}

/* ----- Main ----- */

int main()

```

```

{
    char line[MAX], cmd[MAX], arg[MAX];

    updateWorkingDirectory();

    printf("=== Part 1: Directory Navigation ===\n");
    testNavigation();

    printf("\n=== Part 2: Built-in Command Dispatcher ===\n");
    testDispatchLogic();

    printf("\n=== Interactive Mode (type 'exit' to quit) ===\n");
    while (1)
    {
        updateWorkingDirectory();
        printf("myshell:%s$ ", currDirectory);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0)
            continue;

        arg[0] = '\0';
        sscanf(line, "%s %[^\n]", cmd, arg);

        if (isBuiltin(cmd))
            executeBuiltin(cmd, arg[0] ? arg : NULL);
        else
            printf("%s: command not found (external execution not implemented)\n", cmd);
    }

    return 0;
}

```

Compilation and Execution

1. Compile

```
gcc question9.c -o question9
```

2. Execute

```
./question9
```

Sample Output

```

teja@Hasini:~$ nano question9.c
teja@Hasini:~$ ls
Experiment-2      hello      parser      question7.c
filecopy.c.save  hello.c    parser.c    question9.c
filecopy.c.save.1 linux      path_resolver quotes
fork             os_task    path_resolver.c quotes.c
fork.c           os_task.c  question7
teja@Hasini:~$ gcc question9.c -o question9
teja@Hasini:~$ ls
Experiment-2      hello      parser      question7.c
filecopy.c.save  hello.c    parser.c    question9
filecopy.c.save.1 linux      path_resolver question9.c
fork             os_task    path_resolver.c quotes
fork.c           os_task.c  question7   quotes.c
teja@Hasini:~$ ./question9
=== Part 1: Directory Navigation ===

--- Testing Directory Navigation ---
Starting directory: /home/teja
Directory changed to: /tmp
/home/teja
Directory changed to: /home/teja
cd: /nonexistent_dir_xyz: No such file or directory

=== Part 2: Built-in Command Dispatcher ===

--- Testing Dispatch Logic ---
/home/teja
Directory changed to: /tmp
Built-in commands: cd, pwd, exit, help
foo: command not found
Total commands dispatched: 4

=== Interactive Mode (type 'exit' to quit) ===
myshell:/tmp$ pwd
/tmp
myshell:/tmp$ cd /home
Directory changed to: /home
myshell:/home$ pwd
/home

```

Conclusion

This experiment demonstrates directory navigation using `chdir()`, including path validation with `stat()`, working-directory updates, error handling for invalid paths, and maintenance of the previous directory for "cd -". It also demonstrates a built-in command dispatcher built around a table of function pointers, showing how a shell can identify, execute and manage built-in commands in-process while gracefully handling invalid input. Together, these illustrate important Operating System concepts such as the process working directory, system calls for file-system interaction, and command-dispatch design in shell implementation.